21-03-2026

VT Rushi Kannan

SCALABLE-NLP-CLASSIFICATION REPORT

INTRODUCTION

The objective of this project is to develop a topic classification system that predicts the topic label given input text data. The dataset consists of 10 million samples stored in a compressed Parquet file, making it a large-scale text classification problem.

Due to computational (GPU memory) and time constraints, training on the full dataset was not feasible. Therefore, a representative subset of 100,000 samples was used for experimentation. This allowed efficient model development and comparison across multiple approaches while maintaining reasonable performance evaluation.

Both classical machine learning and deep learning models were implemented from scratch, without using any pretrained models, as per the constraints.

A. DATA PROCESSING

Dataset:

- Total size: 10 million samples (4GB)
- Columns:
    - DATA → input text
    - TOPIC → target label

Data Loading Strategy:

Due to the large dataset size (10 million samples), full data loading was memory-intensive and computationally expensive. Therefore, a subset of 100K samples was extracted for efficient experimentation. This subset was further divided into training (70%), validation (10%), and testing (20%) sets to ensure reliable model evaluation and performance analysis.

Preprocessing Steps:

- Converted text to lowercase
- Removed special characters and noise

- Tokenization applied
- Padding applied for sequence models

Feature Engineering:

- TF-IDF used for ML models
- Token-based vocabulary used for LSTM/BiLSTM
- Vocabulary size limited (e.g., 20,000 words)

B. EXPLORATION & ITERATION

To identify the most effective approach for topic classification, a wide range of machine learning and deep learning models were implemented, evaluated, and iteratively refined.

Initially, the dataset consisted of approximately 10 million samples. However, due to hardware limitations (GPU constraints) and time restrictions, it was not feasible to train deep learning models efficiently on the full dataset. Therefore, a representative subset of 100K samples was used for experimentation.

This decision enabled:

- Faster experimentation cycles
- Multiple model comparisons
- Controlled and reproducible evaluation

The experimentation includes both traditional machine learning models and deep learning architectures, allowing a comprehensive comparison across different paradigms under practical constraints.

Different approaches I tried:

1. Normal Logistic Regression
2. Balanced Logistic Regression
3. Naive Bayes
4. SVM (LinearSVC)
5. FastText
6. LSTM

7. Bi-LSTM

Why each approach was chosen:

1. Normal Logistic Regression

- Chosen as a strong baseline model
- Simple, fast, and highly interpretable
- Works well with TF-IDF features
- Provides a reliable benchmark for comparison

2. Balanced Logistic Regression

- Selected to address class imbalance
- Uses class weighting to improve minority class performance
- Helps evaluate trade-off between accuracy vs fairness

3. Naive Bayes

- Chosen as a lightweight probabilistic baseline
- Extremely fast and scalable
- Commonly used in text classification tasks
- Serves as a reference for simplest possible model

4. SVM (LinearSVC)

- Selected for its strength in high-dimensional sparse data
- Highly effective with TF-IDF representations
- Known for strong generalization performance
- Expected to perform well on structured text classification

5. FastText

- Chosen as a lightweight deep learning model
- Captures word-level semantics efficiently
- Much faster than traditional neural networks
- Suitable for medium-scale datasets (like 100K)

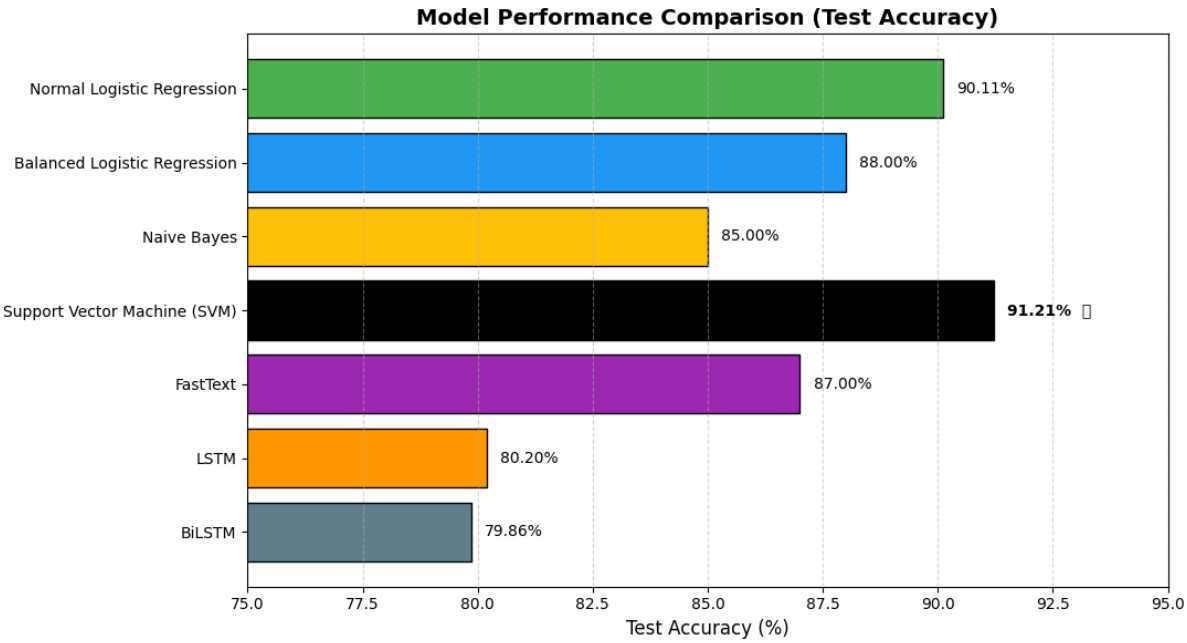
6. LSTM

- Selected to model sequential dependencies in text
- Captures contextual relationships across words
- Represents a standard deep learning baseline for NLP

7. Bi-LSTM

- Chosen to capture bidirectional context (past + future)
- Improves understanding of sentence structure
- Represents a more advanced sequence modeling approach

What worked and what didn't



SVM achieves the highest test accuracy, outperforming both traditional and deep learning models, while LSTM and BiLSTM show comparatively lower performance, indicating that classical ML models are more effective for this task.

1. SVM (LinearSVC)

- Highest accuracy (91.21%)

- Strong generalization (small gap)
- Excellent with TF-IDF features
- Consistent across classes

2. Normal Logistic Regression

- Stable performance (~90%)
- Simple and interpretable
- Reliable across most classes

3. Balanced Logistic Regression

- Improves minority class prediction
- Handles class imbalance well
- Slight drop in overall accuracy

4. FastText

- Best DL model (~87%)
- Fast and efficient
- Strong on frequent classes

What Didn't Work

5. LSTM

- Lower accuracy (~80%)
- Needs more data
- Training is slower

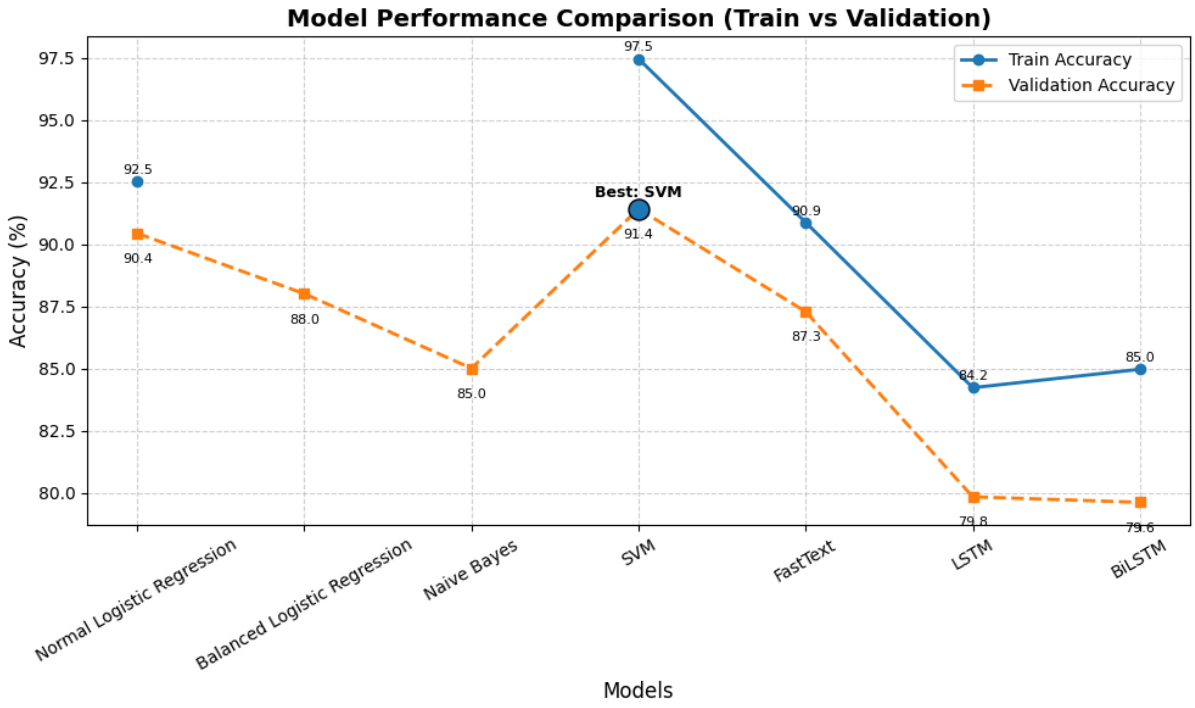
6. Bi-LSTM

- Similar performance to LSTM

- Overfitting observed
- High complexity

7. Naive Bayes

- Moderate accuracy (~85%)
- Assumes feature independence
- Less powerful than other models



SVM shows the best balance between train and validation accuracy, while models like LSTM and BiLSTM exhibit lower validation performance, indicating weaker generalization.

Observations from failed or underperforming models

- LSTM and Bi-LSTM did not perform well, mainly because the dataset was limited (100K). These models usually need much larger data to give better results.
- There was also some overfitting in Bi-LSTM, where training accuracy was higher than validation accuracy.

- Naive Bayes gave average performance. Its assumption that all features are independent is not suitable for real text data.
- Deep learning models in general did not perform as expected due to lack of sufficient data and GPU resources.
- Compared to them, simpler machine learning models handled the task more effectively under these constraints.

How Insights from Earlier Experiments Guided Later Improvements

- Initially, deep learning models like LSTM and Bi-LSTM were tried, but their performance was low. This showed that the dataset size (100K) was not enough for such complex models.
- From this, the focus was shifted to simpler machine learning models, which are known to work well with smaller datasets.
- Early results also showed class imbalance issues, so Balanced Logistic Regression was used to improve minority class performance.
- Observing that text data is sparse and high-dimensional, TF-IDF with linear models (like SVM) was given more importance.
- After comparing all models, SVM was selected because it gave the best accuracy and consistent performance.

Final Model Selection Reasoning

Experiments were done on 100K samples due to resource limits.

Deep learning models (LSTM: 80.20%, Bi-LSTM: 79.86%) performed lower and showed overfitting.
Traditional models performed better:

- Logistic Regression: 90.11%
- Balanced LR: ~88%
- Naive Bayes: ~85%

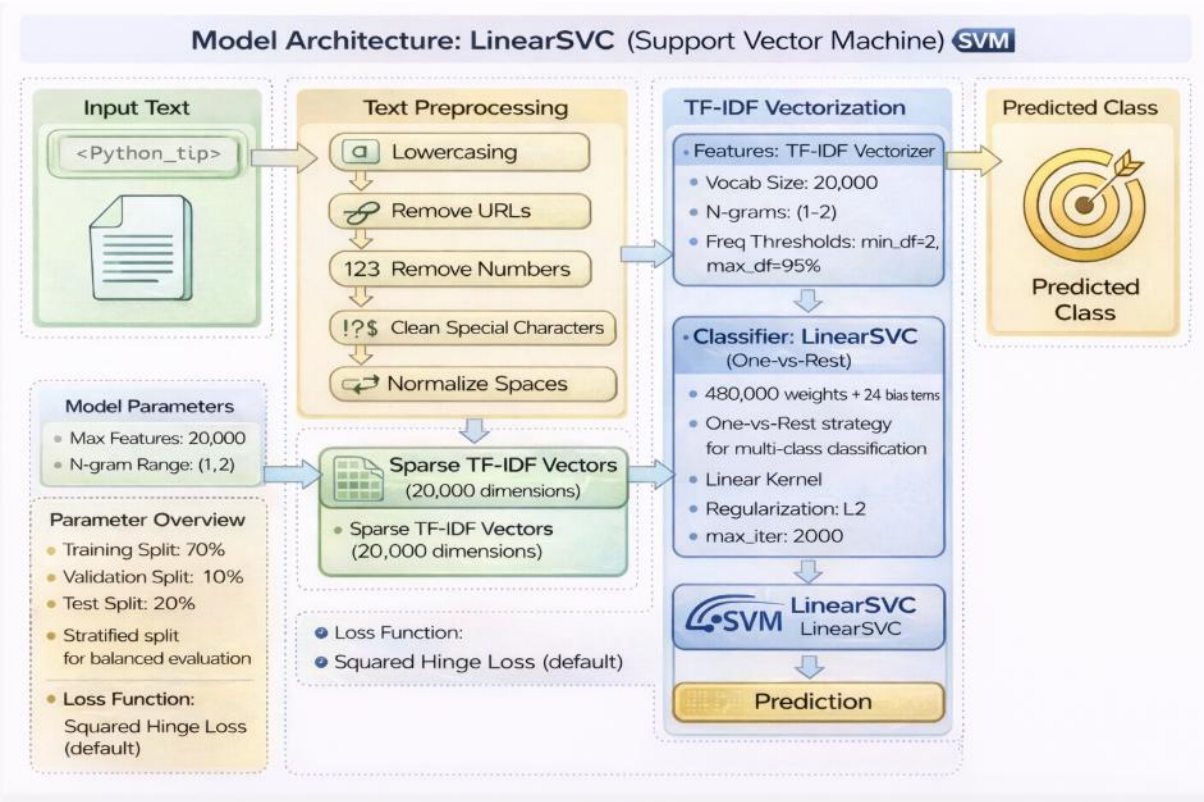
SVM (LinearSVC) gave the best results:

- Train: 97.45% | Val: 91.41% | Test: 91.21%

SVM was selected as it achieved the highest accuracy with good generalization.

C. Architecture Details (Final Model)

Model Structure



- Model: LinearSVC (Support Vector Machine)
- Input: TF-IDF features (max_features = 20,000, n-grams = 1–2)
- Preprocessing: lowercasing, URL removal, number removal, special character cleaning
- Multi-class handling: One-vs-Rest (OvR)
- Pipeline:
  Text → Preprocessing → TF-IDF → LinearSVC → Prediction

Parameter Count

Parameter Count (SVM – LinearSVC)

- Number of Features: 20,000
- Number of Classes: 24

- Total Parameters: 480,000 weights + 24 bias terms

Computation:
20,000 × 24 = 480,000

- This is significantly lower compared to deep learning models (millions of parameters)
- Efficient for handling high-dimensional sparse text data

Design Decisions

- Used TF-IDF (1–2 grams) to capture both individual words and short phrases
- Limited features to 20,000 to balance performance and memory usage
- Selected LinearSVC for handling high-dimensional sparse text data
- Used stratified split (70/10/20) to ensure balanced evaluation

Trade-offs

- Does not capture sequential context like LSTM/Bi-LSTM
- Limited semantic understanding compared to deep learning models
- However, provides:
    - Faster training
    - Lower computational cost
    - Better performance on limited data (100K)

D. Training Strategy

Train / Validation/ Test Split

- Train: 70% | Validation: 10% | Test: 20% (stratified)
- Maintains balanced class distribution across all splits

Hyperparameters

- **Model:** LinearSVC (max_iter = 2000, random_state = 42)

TF-IDF:

- max_features = 20,000

- ngram_range = (1, 2)
- min_df = 2
- max_df = 0.95

Hardware Used

- Training was performed on a CPU-based local system
- No GPU was used due to limited availability
- SVM is computationally efficient and does not require GPU support

Training Time & Efficiency

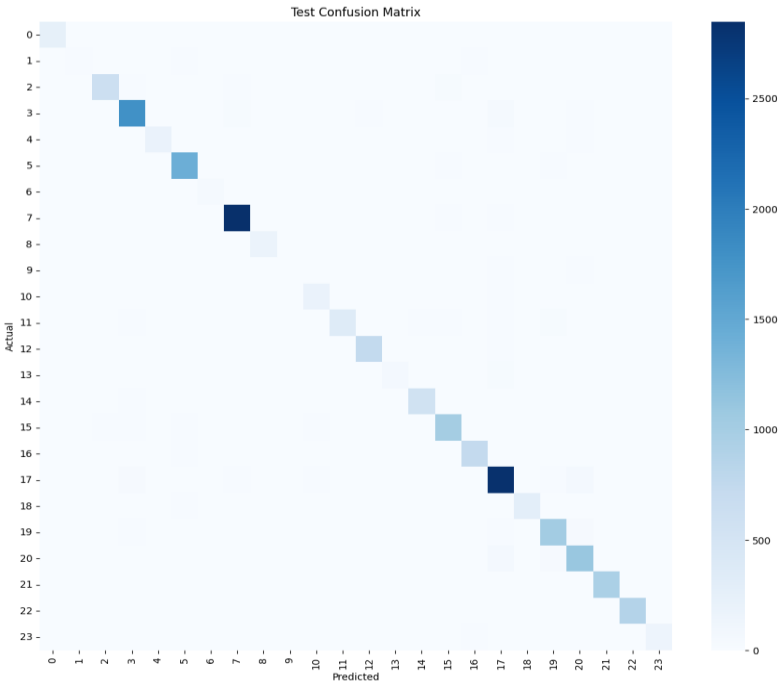
- Training completed in a few minutes on 100K samples
- Efficient due to sparse TF-IDF representation
- Significantly faster than deep learning models, which require GPU and longer training time

The model was designed to balance performance and efficiency, achieving high accuracy with minimal computational resources.

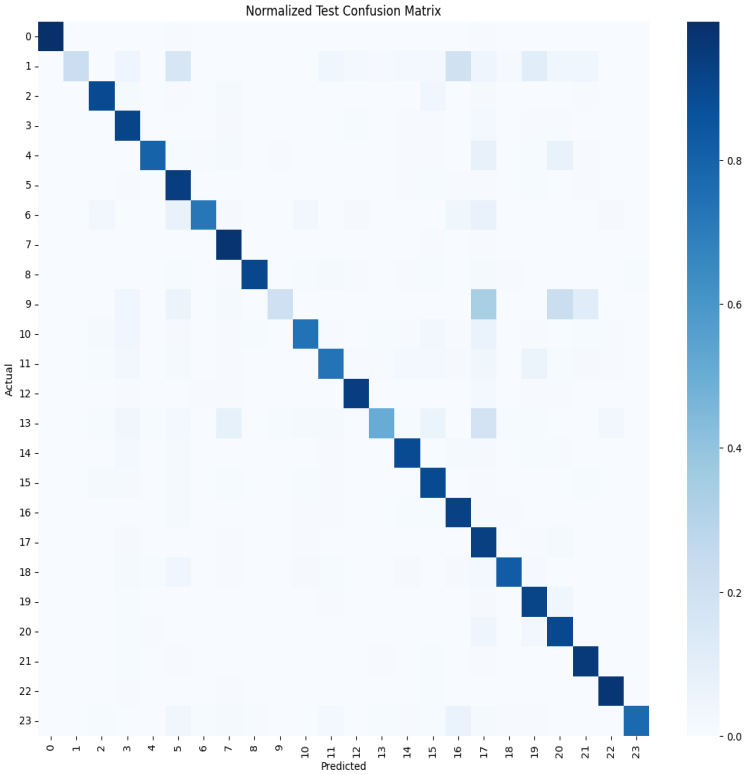
E. Evaluation Metrics

- **Accuracy:**
  Measures overall correctness of the model.
  - Train: **97.45%**
  - Validation: **91.41%**
  - Test: **91.21%**
- **Precision:**
  Measures how many predicted labels are correct.
  - High for most classes (~0.85–0.99)
  - Lower for some minority classes (e.g., class 1, 9)

- **Recall:**
  Measures how many actual instances are correctly identified.
  - Strong for majority classes
  - Lower for minority classes (e.g., recall drops to ~0.21–0.37)
- **F1 Score:**
  Balance between precision and recall
  - Overall weighted F1: **~0.91**
  - Lower for difficult/rare classes



- The confusion matrix exhibits strong diagonal dominance, indicating high classification accuracy, with limited off-diagonal values showing misclassification primarily among minority or similar classes.



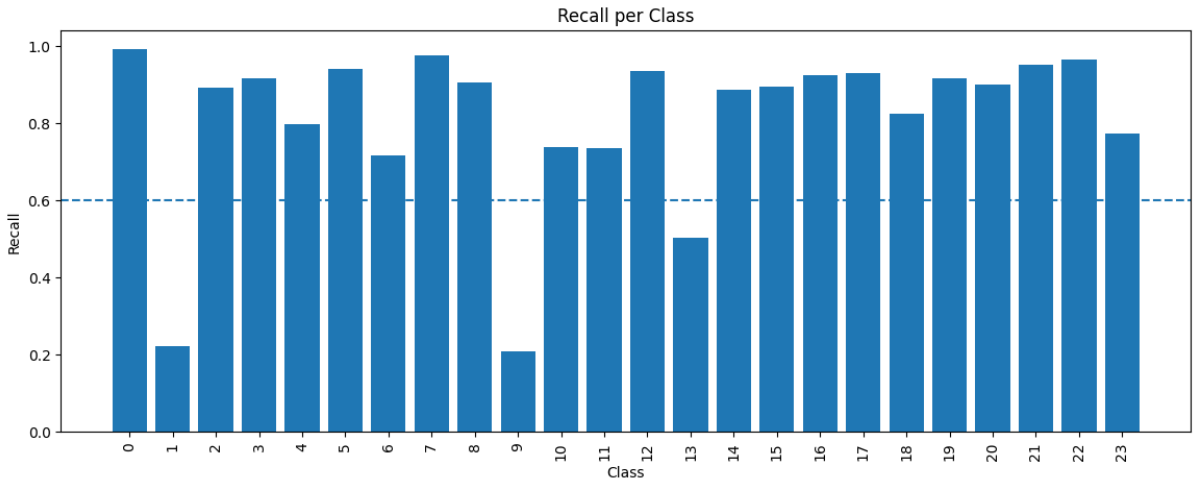
-
- The normalized confusion matrix shows high per-class recall along the diagonal, with noticeable deviations for a few classes, indicating class-wise imbalance and misclassification into dominant categories.

F. Error Analysis

Where the Model Fails

- The model struggles mainly with **low-support classes**, especially:
  - **Class 1** → Recall: **0.22**
  - **Class 9** → Recall: **0.21**
  - **Class 13** → Recall: **0.50**

These classes have **very low recall**, meaning most of their actual samples are misclassified.



The graph shows recall for each class (0–23). Most classes have high recall (>0.8), indicating good performance. However, Class 1, Class 9, and Class 13 have low recall, meaning the model fails to correctly identify many of their samples. This suggests class imbalance and bias toward dominant classes.

Patterns in Misclassification

- **Class 1** is frequently misclassified into:
  - Class 5 (**13 samples**)
  - Class 16 (**16 samples**)
  - Class 19 (**9 samples**)
- **Class 9** is often confused with:
  - Class 17 (**18 samples**)
  - Class 20 (**12 samples**)
- **Class 3 (large class)** shows confusion with:
  - Class 17 (**50 samples**)
  - Class 7 (**24 samples**)
- **Class 17 (major class)** absorbs many misclassifications from other classes
  → Indicates **bias toward majority classes**

Overall pattern:

- Minority classes → predicted as **majority classes (like 7, 17, 20)**
- Similar topic classes → overlap in predictions

Potential Improvements

- Increase dataset size from 100K to the full 10M samples to better utilize deep learning models
- Use advanced loss functions such as:
    - Focal Loss → to handle class imbalance
    - Dice Loss → to improve performance on minority classes
- Apply class balancing techniques (class weights / oversampling) to further reduce bias toward majority classes
- Explore deep learning architectures such as LSTM, Bi-LSTM, and GRU with proper hyperparameter tuning
- Train models using GPU-based systems, as current experiments were limited by CPU, time constraints, and reduced dataset size